

clautolisp path / pathname mapping for cross-environment MS-Windows

June 23, 2026

Contents

1	1. Problem statement	1
1.1	1.1 Three frames of reference	2
1.2	1.2 Why we cannot "just use logical pathnames"	3
2	2. Goal	3
3	3. Definitions	4
4	4. Architecture	4
4.1	4.1 Environment + mount-table model	4
4.2	4.2 Two primitives	4
4.3	4.3 Insertion points	5
5	5. Detection	5
5.1	5.1 Build frame (record at build time)	5
5.2	5.2 Run frame (detect at startup)	6
6	6. Edge cases and rules	6
7	7. Optional extension: opt-in CL logical pathnames	7
8	8. Conformance test matrix	7
9	9. Implementation phases	7

- **Status:** design specification (problem statement + proposed architecture). No implementation yet.
- **Scope:** MS-Windows builds and runs of clautolisp across coexisting environments — native Windows, Cygwin, MSYS2, and WSL. macOS and Linux keep the trivial identity mapping.
- **Tracks:** [issues/open/clautolisp-windows-pathname-mapping.issue](#).
- **Related (resolved):** [issues/closed/clautolisp-boot-cwd-pwd-pathname-defaults.issue](#) (the cwd/PWD base at boot — a special case handled in 1.2.8). [issues/open/cad-path-dotdot-resolution.issue](#) (`..` semantics once the base and rooting are correct).

1 1. Problem statement

The AutoLISP / AutoCAD / BricsCAD API works with **path strings** — plain strings such as `"C:/work/dwg/plan.lsp"` or `"../lib/util.lsp"`. clautolisp must accept and return those strings; it cannot expose Common Lisp **pathname** objects to user code. Internally, however, it resolves files through the host CL implementation (SBCL or CCL), which works in terms of CL **pathname** objects and the **physical** path syntax / roots of the operating environment it was built for.

On macOS and Linux this is a non-problem: the CL implementation is well-behaved, the root is unambiguously `/`, **merge-pathnames** does the right thing, and a single environment is in play. The string the user passes, the string the OS understands, and the pathname the CL runtime builds all agree. The mapping is the **identity**.

On MS-Windows there is **not a single environment**. Several environments coexist, each with its own path syntax and its own virtual mount points / roots for the **same physical files**:

Environment	Root syntax for the C: volume	Example path
Native Windows	drive letters, <code>C:\</code> or <code>C:/</code>	<code>C:/Users/pjb/x.lsp</code>
Cygwin	POSIX, <code>/cygdrive/c/</code> (configurable)	<code>/cygdrive/c/Users/pjb/x.lsp</code>
MSYS2	POSIX, <code>/c/</code> (MSYS2 mount table)	<code>/c/Users/pjb/x.lsp</code>
WSL (1/2)	POSIX, <code>/mnt/c/</code>	<code>/mnt/c/Users/pjb/x.lsp</code>
MSYS2 / seen from native Windows	a real directory, e.g. <code>C:/msys64/</code>	<code>C:/msys64/home/pjb/x.lsp</code>

`~/` (home) likewise resolves to different physical directories depending on the environment (e.g. `C:/Users/pjb` natively vs `C:/msys64/home/pjb` under MSYS2).

1.1 1.1 Three frames of reference

At run time there are (at least) three frames that must be reconciled for every path string crossing the API boundary:

1. **Build frame** — what the host CL implementation expects as physical pathnames: the syntax and the roots / mount points of the environment **SBCL/CCL was compiled in**. This is baked into the dumped image.
2. **Run frame** — the Windows environment clautolisp is **actually running under** now (native / Cygwin / MSYS2 / WSL), with its own mount table.
3. **User frame** — the syntax and rooting of the path strings the **user code** passes in (and expects back), which can be different again.

The hard cases come from these frames disagreeing:

- An image **compiled** under MSYS2 (build frame = MSYS2 POSIX roots) is **run** under native Windows (run frame = drive letters). The host CL may still believe `/c/...` means something, while the user types `C:/....`.
- **WSL is especially nasty**: `uname` says `Linux`, so naive OS detection classifies it as the identity-mapping Linux case, yet the surrounding Windows volumes are reachable as `/mnt/c/...` and a user may hand in a native `C:\...` path expecting it to work.

1.2 1.2 Why we cannot "just use logical pathnames"

CL **logical pathnames** are the textbook answer to "indirect names -> physical files", but they do not fit here:

- The AutoLISP/AutoCAD/BricsCAD API passes **plain path strings** and expects path strings back. A logical-pathname host ("`CLAL:lib;util.lsp`") is a different, non-portable surface that user code does not speak.

- Logical pathnames standardise an *indirection* namespace; they do not solve *which physical syntax and mount table* the runtime should target.

Exposing CL logical pathnames as an **opt-in clautolisp extension** layered on top of the string API is a separate, optional idea (see §7); it does not replace the mapping layer specified here.

2 2. Goal

Provide a **path / pathname mapping layer** — conceptually similar to a mount table — that is transparent to user code and that:

- knows the **build frame** (1): the physical-pathname syntax and roots the underlying CL implementation was built for;
- knows the **run frame** (2): the path syntax and mount points of the active runtime environment;
- **translates** path strings between the **user frame** (3) and whatever the CL runtime needs in order to hit the correct physical file, in both directions:
 - **in**: user/AutoLISP string -> CL pathname the host can open;
 - **out**: CL pathname/string from the host -> user/AutoLISP string in the syntax the user expects.

Non-goals: changing the AutoLISP string API; doing any work on macOS/Linux beyond the identity mapping; emulating volume semantics the host OS does not provide.

3 3. Definitions

path the POSIX / AutoCAD / BricsCAD notion — a plain string.

pathname the Common Lisp structured object.

environment a (syntax, mount-table, home-resolution) triple: **:native-windows**, **:cygwin**, **:msys2**, **:wsl**, **:posix** (macOS/Linux).

mount entry a rule mapping a path prefix in one environment to the equivalent physical prefix (e.g. **/mnt/c/** <-> **C:/**).

canonical physical path the native-Windows drive-letter form (C:/..., forward slashes) used as the internal pivot all environments translate to and from. Chosen because it is the form the NT kernel and every Windows CL build ultimately resolve to.

4 4. Architecture

A single canonicalisation/translation step is applied to **every path string crossing the API boundary**, driven by the (build-frame, run-frame) pair. All translation pivots through the **canonical physical path** (§3):

```
user string  --[parse in run/user frame]--> canonical physical path
canonical physical path --[render in build frame]--> CL pathname --> host open/load
host result (CL pathname) --[->canonical]--> canonical physical path
canonical physical path --[render in user/run frame]--> user string
```

4.1 4.1 Environment + mount-table model

- A mapping-environment structure: kind, separator, mount-table (ordered list of mount entries, longest-prefix-wins), home resolver, drive-style (drive-letter vs rootless).
- Two live instances held in the image: ***build-environment*** and ***run-environment***. On **:posix** both are the identity environment and the whole layer short-circuits.

4.2 4.2 Two primitives

- **map-in (string) -> pathname**: parse string in the **user/run** frame to a canonical physical path, then render a CL pathname in the **build** frame for the host to open.
- **map-out (pathname-or-string) -> string**: take a host pathname (build frame), canonicalise, then render a path string in the **user/run** frame.

Both are pure functions of the two environments; both reduce to identity on **:posix**.

4.3 4.3 Insertion points

The layer wraps the **physical-access boundary only**, never user-visible string manipulation (`strcat`, `vl-filename-*` stay string-only):

- `resolve-open-pathname` / `resolve-load-pathname` / `resolve-existing-file` (`clautolisp/autolisp-builtins-core/source/api.lisp`) call `map-in` on the resolved string before handing it to the host.
- `findfile` and any builtin returning a path to user code call `map-out`.
- The boot cwd sync (`synchronize-process-cwd` in `tools/clautolisp/source/main.lisp`) records the run-frame cwd in canonical form. This is the §1 base-reference problem, already solved for the POSIX case in 1.2.8 and generalised here.

5 5. Detection

5.1 5.1 Build frame (record at build time)

The environment SBCL/CCL was built for is fixed once the image is dumped, so **detect it at build time and store it in the image**:

- A build step probes the compiling environment (e.g. via `MSYSTEM`, `CYGWIN`, `uname -o`, the presence of `/proc/version` mentioning `Microsoft=WSL`, and the CL implementation's own `*default-pathname-defaults*` / (`trueuname "/"`)) and emits a small `build-environment` descriptor compiled into the image.
- Rationale: (`trueuname ...`) at build time reflects the build frame, which is exactly what must be remembered (cf. the dump-time cwd bug fixed in 1.2.8 — same class of "value captured at dump time", but here it is the value we **want** to keep).

5.2 5.2 Run frame (detect at startup)

At process start, classify the active environment and build its mount table:

- `:wsl` — `/proc/version` or `/proc/sys/kernel/osrelease` contains `microsoft / WSL`; mount table seeded from `/mnt/<letter>/` and, on WSL2, the parsed output of `mount` (the `9p/drvfs` entries).

- `:cygwin / :msys2` — `MSYSTEM / OSTYPE`, plus the `cygwin/msys2` mount table (`/etc/fstab` + built-in `/cygdrive` or `/<letter>` prefixes).
- `:native-windows` — neither of the above and the OS is Windows; drive-letter roots, separators normalised `\ -> /` internally.
- `:posix` — macOS/Linux that is **not** WSL; identity environment.

The classification is **independent** of `uiop:os-windows-p` alone, precisely because WSL reports Linux (the §1.1 trap).

6 6. Edge cases and rules

- **Already-canonical input** — a `C:/...` string under a POSIX-rooted run frame is still mapped to the build frame; never assume "looks native => pass through".
- **UNC paths** (`//server/share/...`) — preserved as a distinct root class; no drive-letter rewriting.
- **Mixed separators** — normalise to `/` internally; render with the target environment's separator on the way out.
- **~/ and relative paths** — `~` resolved per the run frame's home; relative paths resolved against the canonical cwd (§4.3) before mapping, so this layer composes with the boot-cwd fix and with `..` resolution (cad-path-dotdot-resolution.issue).
- **Unmappable path** — a path with no mount entry in the target frame is an explicit error (a `sysvar-visible` / `*error*-visible` condition), not a silent wrong-file open.
- **Round-trip stability** — `map-out(map-in(s))` must equal the canonical form of `s` in the user frame for every path the user can express; covered by the conformance matrix (§8).

7 7. Optional extension: opt-in CL logical pathnames

Separately from (and on top of) the string API, `clautolisp` **may** expose CL logical pathnames as a documented `clautolisp` extension for power users

who want host-portable indirection. This is optional, additive, and out of scope for the mapping layer’s correctness; it is recorded here only so the two ideas are not conflated.

8 8. Conformance test matrix

A dedicated test matrix exercises every (**build-frame** x **run-frame**) pair that can occur in practice, each with: an absolute native path, an absolute POSIX path, a ~/ path, a relative path, a UNC path, and a no-mount-entry path.

build \ run	native	cygwin	msys2	wsl1	wsl2
native	ok	ok	ok	ok	ok
msys2	ok	ok	ok	ok	ok
cygwin	ok	ok	ok	ok	ok
wsl (linux build)	-	-	-	ok	ok

Each cell asserts: **map-in** opens the right physical file, **map-out** renders the expected user-frame string, and the round-trip is stable. The matrix can only be **run** on a real Windows host (or a CI runner per environment), which is why this work is gated on Windows hardware — see §10.

9 9. Implementation phases

1. **W0 — model + identity.** mapping-environment, map-in~/~map-out, *build-environment*~/~*run-environment*; identity on :posix. Wire the §4.3 insertion points to call through the (identity) primitives. No behaviour change on macOS/Linux; pure refactor guarded by tests.
2. **W1 — build-frame detection.** Build step records the build environment in the image; expose it (debug-only) for inspection.
3. **W2 — run-frame detection.** Startup classifier + mount-table construction for native / cygwin / msys2 / wsl.
4. **W3 — translation.** Implement non-identity map-in~/~map-out pivoting on the canonical physical path; edge-case rules (§6).
5. **W4 — conformance matrix.** §8 across CI runners per environment.
6. **W5 — optional logical-pathname extension** (§7), if pursued.

10 10. Why this is deferred (not yet implemented)

The narrow, testable subset that clautolisp could fix off-Windows — relative paths resolving against the live cwd at boot — was split out and **fixed in 1.2.8** (boot-cwd issue). Everything remaining here is intrinsically cross-environment Windows behaviour: it cannot be reproduced or verified on the Linux/macOS build hosts, where the mapping is the identity and all matrix cells are trivially green. Implementing W1–W4 without a real Windows target (ideally one runner per native / Cygwin / MSYS2 / WSL) would be writing code that no test here can exercise. The layer is therefore specified now and scheduled against Windows hardware.